Distracted Driver Detection from Dashboard Images

UE24CS352A Machine Learning, Mini-Project (Problem #24) | Arya Devang (PES1UG24AM054) | Ananthakrishna (PES1UG24AM036)

Code: github.com/Arya-677/ML_miniProject_sem5 (private; shared with faculty/TAs)

1. Problem statement

Given one in-cabin camera image of a driver, predict what the driver is doing: safe driving or one of nine distractions (texting or talking on the phone with the left or right hand, operating the radio, drinking, reaching behind, hair/makeup, talking to a passenger). This is a 10-class image classification problem. We re-implement the CS229 report *Machine Learning Techniques for Distracted Driver Detection* (Feng & Yue, 2019), which compares five classifiers on raw pixels, and extend it with a stricter question: **does the accuracy carry over to drivers never seen in training?**

2. Dataset

State Farm Distracted Driver Detection (Kaggle): 22,424 labelled 640x480 RGB images, 10 classes (c0 safe driving, c1-c9 distractions), roughly 2,000-2,500 per class, from **26 drivers**; `driver_imgs_list.csv` gives the driver of every image. The images are frames of short videos, so frames of one driver are near-duplicates. The official test images are unlabelled, so we hold out 20% of the labelled images for validation.

3. Approach

- **Pre-processing:** resize 640x480x3 to 64x64x3 (OpenCV, area interpolation; 75x fewer inputs), flatten to a 12,288-vector, scale to [0,1], subtract the mean image computed on training data only.
- **Two evaluation protocols:** (A) random 80/20 split, as in the reference report (17,940 train / 4,484 val; all 26 drivers on both sides); (B) **driver-wise** 80/20 split with whole drivers held out (17,747 train / 4,677 val; 21 / 5 drivers, no overlap), which measures generalisation to new people.
- **Tuning without leakage:** learning rate and regularisation are chosen by grid search on an inner split of the training data, so the validation set is used only for the final numbers.

Model	Built	Loss / structure	Distinctive feature
Linear SVM	NumPy	Scores XW+b, multi-class hinge loss (margin 1) + L2	Only margin violators produce gradient; linear boundary
Softmax	NumPy	Same linear scores, cross-entropy + L2	Outputs class probabilities; every sample contributes
Two-layer NN	NumPy	12,288 - 100 ReLU - 10 softmax (1.23M weights)	Non-linear features; He init; learning-rate decay 0.9/epoch
Naive Bayes	scikit-learn	Gaussian per pixel and class, independence assumed	No iterative training; ignores pixel correlations
Decision Tree	scikit-learn	Axis-aligned pixel-threshold splits, default settings	Unpruned: 100% training accuracy

4. Implementation overview

Everything lives in one notebook (`distracted_driver_detection.ipynb`): data download and caching, vectorised losses and back-propagation in NumPy, mini-batch SGD (batch 256 for SVM/softmax, 200 for the network, 2,000 iterations, fixed seed 0 because SVM and softmax are sensitive to weight initialisation), the grid search (4x3 settings for SVM and softmax, 3x3 for the network), evaluation (accuracy, per-class accuracy, confusion matrix, loss curves) and saved models. **Verification:** a numerical gradient check confirms all analytic gradients (max error 2.1e-10 for SVM, 7.4e-11 for softmax, below 6e-11 for the network). A **live-demo cell** loads the saved network and prints the predicted activity with the top-3 probabilities for any image. The repository also holds `README.md` (setup and run instructions), `requirements.txt`, and `results_random/` and `results_driver/` (results, plots, models).

5. Results

Model	Random: train	Random: val	Paper (random)	Driver-wise: train	Driver-wise: val
Naive Bayes	55.47	55.22	54.99	61.12	32.61
Decision Tree	100.00	89.12	84.73	100.00	29.66
Linear SVM	96.16	95.34	71.39	93.63	45.39
Softmax	71.69	71.39	82.31	75.97	41.93
Two-layer NN	98.57	98.02	92.24	99.10	51.55

Accuracy (%). Best validation result of each protocol is shaded. The Paper column is the CS229 report (random split) for comparison.

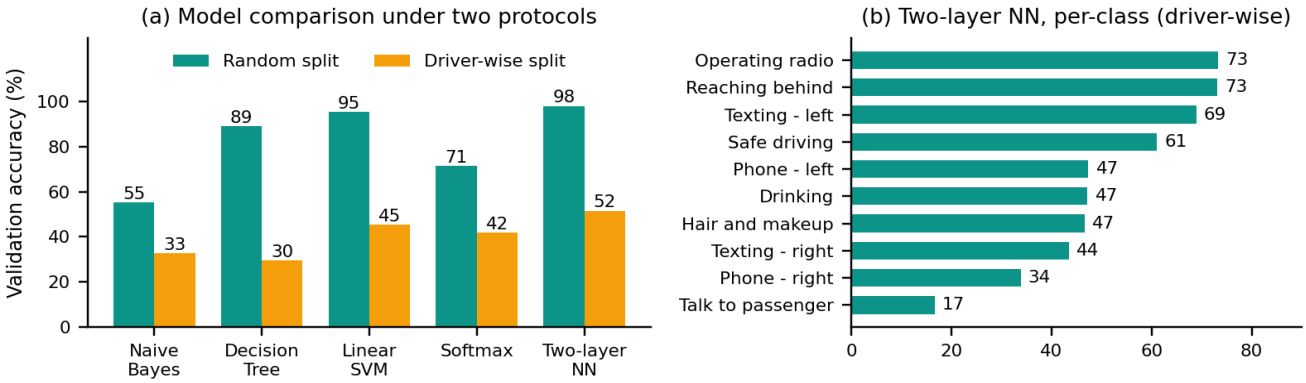
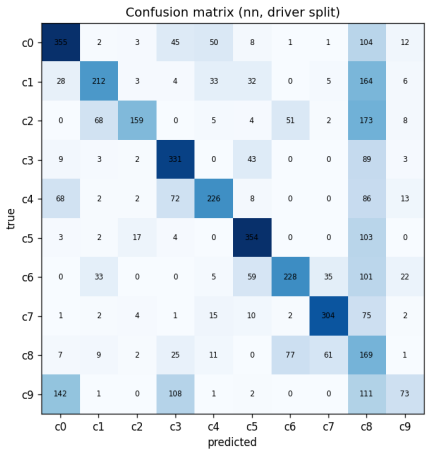


Figure 1. (a) Validation accuracy of all five models under both protocols. (b) Per-class accuracy of the best model on unseen drivers.

- **Ranking:** the two-layer network is best under both protocols (98.0% random, 51.6% driver-wise). On the random split the ranking is NN, SVM, tree, softmax, Naive Bayes; on unseen drivers it is NN, SVM, softmax, Naive Bayes, tree.
- **Random-split scores are optimistic:** holding out whole drivers lowers accuracy by 41.6 points on average. The tree falls furthest (89.1% to 29.7%; 100% training accuracy), and the network fits training data at 99.1% against 51.6% validation, a 47.6-point gap. Near-duplicate frames let models memorise people (clothes, face, car interior) rather than activities.
- **Versus the paper:** our network (98.0 vs 92.2), tree (89.1 vs 84.7) and Naive Bayes (55.2 vs 55.0) match or exceed it. Our SVM is much higher (95.3 vs 71.4) because we tuned it by grid search with a fixed initialisation, which the paper found critical; our softmax is lower (71.4 vs 82.3).



- **Error analysis (network, unseen drivers).** Best classes: operating the radio (73.3%) and reaching behind (73.1%). Worst: talking to a passenger (16.7%) and talking on the phone with the right hand (33.8%).
- **Hair and makeup is a catch-all:** column c8 of the matrix receives 25% of all predictions (1,175 of 4,677) although the class is only 7.7% of the data. The largest single confusion is phone-right predicted as hair/makeup (173 images).
- **Why:** phone-right, texting-right, drinking and hair/makeup share one pose (right hand near the face), and at 64x64 the object in the hand is only a few pixels. Talking to a passenger has no distinctive object at all.

Figure 2. Confusion matrix of the two-layer network on the driver-wise validation split (rows true, columns predicted).

6. Conclusions and limitations

A small from-scratch two-layer network on raw pixels is the best of the five models, reaching 98.0% when the same drivers appear in training and validation, but only 51.6% on new drivers; the random-split figure therefore overstates real-world performance. Limitations: 64x64 pixels discard detail and fully-connected models ignore spatial structure; results use one seed and one split, and the driver-wise validation set has only 5 drivers, so the numbers are noisy. **Next steps:** a CNN or transfer learning, data augmentation, driver-grouped k-fold cross-validation, and semi-supervised use of the unlabelled test images.